

CQt day4 hw2 레포트(UDP+hw1 보고서)

로봇 예비단원 주호진

1. 과제

DP 조사 보고서 + 과제 1번 보고서

- 합쳐서 전체 15p 이상 작성
- UDP 조사 보고서의 경우 자신이 UDP에 대해 무엇을 공부했고 배웠는지 작성
- 과제 1번 보고서의 경우 자신이 Qt에서 UDP를 어떻게 적용시켰는지
관련 내용 반드시 포함할 것

2. UDP(User Datagram Protocol)

컴퓨터가 다른 컴퓨터와 데이터 통신을 하기 위한 프로토콜의 일종

*OSI 7계층 중 4계층에 해당함

*OSI 7계층

응용 계층	DHCP, FTP, DNS, HTTP, POP, SMTP	응용계층
표현 계층		
세션 계층		
전송 계층	TCP UDP Segment	전송계층
네트워크 계층	IP Address : IPv4 IPv6 Datagram	인터넷 계층
데이터링크 계층	MAC Address Frame	네트워크 접근 계층
물리 계층	Ethernet cable, wire...	

OSI 표준 모델

TCP/IP 모델

- ISO에서 개발한 모델
- 컴퓨터 네트워크 프로토콜 디자인과 통신을 계층으로 나누어 개념적으로 설명한 것
- 4계층은 전송 계층에 해당

2-1. 개발 배경

기존 통신 간 누락된 데이터를 모두 받을 수 있는 매커니즘이 정의된 TCP 통신 사용함

-> 신뢰성은 보장이 되나 실시간 송출과 같은 서비스에서 문제가 됨

-> 데이터 신뢰도를 포기하는 대신 속도가 빠른 UDP가 제시 됨

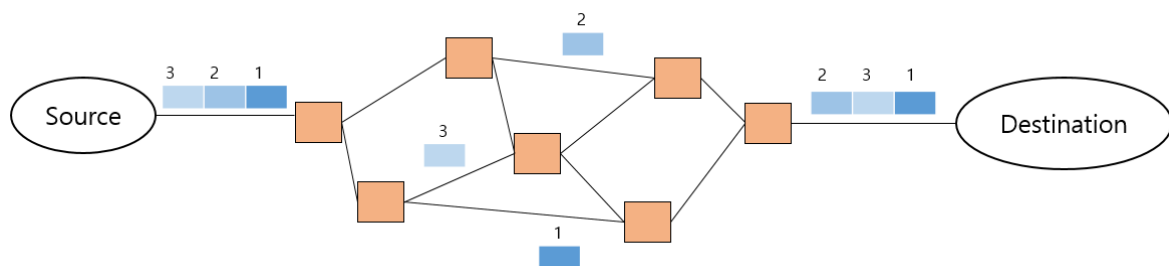
2-2. UDP 특징

1) Receiver의 준비 상황을 고려하지 않음

Sender는 오직 보내기만 함, receiver가 준비되지 않았다면 패킷들이 유실되거나 전송되지 않음

2) 패킷이 제대로 전송되었는지 확인하지 않음

3) 패킷의 순서를 상관하지 않고 보냄



도착 순서를 보장하지 않기 때문에 수신 측에서 데이터를 재정렬하는 과정이 필요함

4) CheckSum 필드를 통해 최소한의 오류만 검출

5) TCP에 비해 빠른 속도

TCP에서 진행하는 오류 검사 및 재전송 기능이 없기 때문에 빠름

6) 낮은 신뢰성

	TCP	UDP
연결 방식	연결형 서비스	비연결형 서비스
패킷 교환	가상 회선 방식	데이터그램 방식
전송 순서 보장	보장함	보장하지 않음
신뢰성	높음	낮음
전송 속도	느림	빠름

+TCP도 UDP처럼 검사 옵션을 꺼 속도를 올릴 수 있지만, 패킷 자체가 UDP 형식보다 커서 느림
 ++(UDP + 커스텀)을 통해 낮은 신뢰도를 보강하는 방법으로 사용하기도 함

TCP에 비해 신뢰성은 낮지만, 속도가 빠르므로 속도가 중요한 작업에서 사용됨

Ex) 실시간 송출, 빠른 전송이 필요한 대용량 파일 등

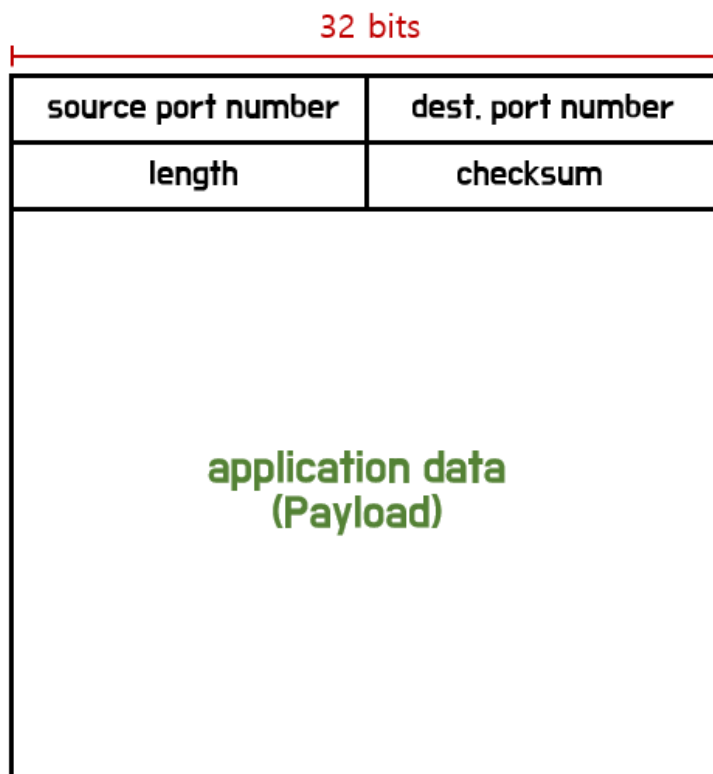
위 예시의 경우 데이터의 일부 유실보다 실시간으로, 혹은 빠른 시간 내에 보내는 목적이 더 중요하기 때문에 TCP를 사용

생방송(실시간 송출)의 경우 방송의 몇 프레임이 깨지는 것보다, 버퍼링이 걸리는 것이 더 불편한 상황이기 때문임

2-3. UDP 데이터그램

source Port number, Dest. Port number, length, checksum가 담긴 헤더(8Byte)와 data가 담긴 payload가 있음

이를 합하면 datagram이라는 패킷이 됨



- source Port number

패킷을 전송하는 발신 측의 포트 번호로, 수신 측이 응답을 보낼 때 사용

- Dest. Port number

패킷을 받아야 하는 수신 측 포트 번호

- length

UDP 헤더와 페이로드를 포함한 전체 패킷의 크기를 나타냄

- checksum

치명적 오류를 피하기 위해 검사를 담당, 그러나 오류가 발견되어도 폐기하고 재전송은 하지 않음

- Payload

데이터가 담긴 곳

2-4. QUdpsocket Class(QT)

Qt기반 UDP 통신을 할 수 있게 해주는 class

사용 방법

- Bind()를 통해 주소와 포트를 지정

```
void Server::initSocket()
{
    udpSocket = new QUdpSocket(this);
    udpSocket->bind(QHostAddress::LocalHost, 7755);

    connect(udpSocket, &QUdpSocket::readyRead,
            this, &Server::readPendingDatagrams);
}
```

- 기본적으로 writeDatagram()과 readDatagram()/receiveDatagram()을 통해 data를 주고 받음

QNetworkDatagram QUdpSocket::receiveDatagram(**qint64** maxSize = -1)

다음보다 크지 않은 데이터그램을 수신합니다. *maxSize*바이트 단위로 변환하여 반환합니다. **QNetworkDatagram** 이 함수는 송신자의 호스트 주소와 포트와 함께 데이터그램 객체를 반환합니다. 가능하면 이 함수는 수신 시 데이터그램의 목적지 주소, 포트 및 홉 카운트도 확인하려고 시도합니다.

실패 시, 반환값 **QNetworkDatagram** 그 보고서는 **not valid**.

만약에 *maxSize*크기가 너무 작으면 나머지 데이터그램이 손실됩니다. *maxSize*값이 0이면 데이터그램은 폐기됩니다. *maxSize*값이 -1(기본값)이면 이 함수는 전체 데이터그램을 읽으려고 시도합니다.

관련 항목도 참조하세요 **writeDatagram()**, **hasPendingDatagrams()**, 그리고 **pendingDatagramSize()**.

ReceiveDatagram은 호출하면 Datagram을 받아옴, 매개변수는 기본값 -1이고, -1은 데이터그램을 읽으려고 시도하는 값

qint64	writeDatagram (const char *data, qint64 size, const QHostAddress &address, quint16 port)
qint64	writeDatagram (const QNetworkDatagram &datagram)
qint64	writeDatagram (const QByteArray &datagram, const QHostAddress &host, quint16 port)

WriteDatagram은 파라미터를 받는 종류가 3가지인데, ppt 예제에서 datagram 형식인 text_socket을 선언하고 있으므로 세번째 줄에 있는 writeDatagram을 사용

qint64 QUdpSocket::writeDatagram(const **QByteArray** &datagram, const **QHostAddress** &host, **quint16** port)

데이터그램을 전송합니다. *datagram*호스트 주소로 *host*그리고 항구에서 *port*.

이 함수는 성공하면 전송된 바이트 수를 반환하고, 오류가 발생하면 -1을 반환합니다.

WriteDatagram은 datagram과 수신받을 IP, PORT를 매개변수로 입력해주면 send해줌
이때 송신에 성공하면 바이트 수를 반환, 실패하면 -1 반환

- datagram이 도착하면 hasPendingDatagrams()가 true를 반환

```
void Server::readPendingDatagrams()
{
    while (udpSocket->hasPendingDatagrams()) {
        QNetworkDatagram datagram = udpSocket->receiveDatagram();
        processTheDatagram(datagram);
    }
}
```

따라서 위 예제와 같이 hasPendingDatagram이 true를 반환할 때 datagram을 받아서 사용할 수 있음

3. Hw1 보고서

3-1. 코드 구조

3-1-1. UDP

CMakeList.txt

```
41
42 | find_package(Qt6 REQUIRED COMPONENTS Widgets Network)
43 | target_link_libraries(${PROJECT_NAME} PRIVATE
44 |     Qt6::Widgets
45 |     Qt6::Network
46 | )
47
```

UDP 구현을 위해 CMakeList에 Qt Network를 추가

Mainwindow.h

```
~MainWindow();

QUdpSocket *text_socket = new QUdpSocket(this);

void udp_read();
QString current_sender;

private slots:
void on_sendBtn_clicked();
```

UDP 통신에서 WriteDatagram에 넣을 QUdpsocket 형식인 socket을 선언, 현재 문자를 보내고 있는 객체를 판단할 current_sender, udp값을 읽고 쓰는 read 함수, write(버튼 slot과 연결)을 선언

Mainwindow.cpp

```
uint16_t TEXT_PORT = 10004;
QHostAddress ROBOT_IP = QHostAddress("172.100.0.183");
QHostAddress OPERATOR_IP = QHostAddress("172.100.6.53");
```

메세지를 주고 받을 PORT와 IP를 선언 (PORT번호는 1024~49151번 중 임의의 번호로 설정)

```
MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent)
, ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    if(text_socket->bind(OPERATOR_IP, TEXT_PORT))
    {
        connect(text_socket, &QUdpSocket::readyRead, this, &MainWindow::udp_read, Qt::UniqueConnection);
    }
    else {
```

실행 후 바로 bind 설정이 되도록 함

readyRead(hasPendingDatagrams을 true로 반환해주는) signal을 read 함수와 connect해줌

```

void MainWindow::udp_read() {
    while (text_socket->hasPendingDatagrams()) {
        qDebug() << "수신됨";
        QNetworkDatagram datagram =
            text_socket->receiveDatagram();

        QByteArray data = datagram.data();

        QString message = QString::fromUtf8(data);

        // 상대방 메시지 출력
        QTextCursor cursor = ui->textEdit->textCursor();
        cursor.movePosition(QTextCursor::End);

        QTextBlockFormat blockFormat;
        blockFormat.setAlignment(Qt::AlignLeft);
        cursor.setBlockFormat(blockFormat);

        if(current_sender != QString("상대"))
            cursor.insertText(QString("상대\n"));

        current_sender = QString("상대");
        cursor.insertText(QString("%1\n")
            .arg(message));
    }
}

```

hasPendingDatagram이 true를 반환하면(데이터 패킷이 감지되면) 해당 데이터를 받아옴

받은 datagram에서 data는 ui상의 textBox에 출력하여주고, 이때 이전에 메시지를 한 sender와 동일하지 않은 경우 닉네임을 표시해주도록 함("상대", "나"로 구분)

qint64 QUdpSocket::pendingDatagramSize() const

대기 중인 첫 번째 UDP 데이터그램의 크기를 반환합니다. 사용 가능한 데이터그램이 없으면 -1을 반환합니다.

만약 수신은 되는데 데이터가 뜨지 않는 경우

pendingDatagramSize를 통해 data가 비었는지, 있는지 확인해볼 수 있음


```

void MainWindow::on_sendBtn_clicked()
{
    QString message = ui->plainTextEdit->toPlainText();
    QByteArray data = message.toUtf8();

    qint64 sentBytes = text_socket->writeDatagram(data, ROBOT_IP, TEXT_PORT);
    if (sentBytes == -1) {
        qDebug() << "전송 실패:" << text_socket->errorString();
    } else {
        qDebug() << "전송 성공";
    }

    // 내 메시지
    QTextCursor cursor = ui->textEdit->textCursor();
    cursor.movePosition(QTextCursor::End);

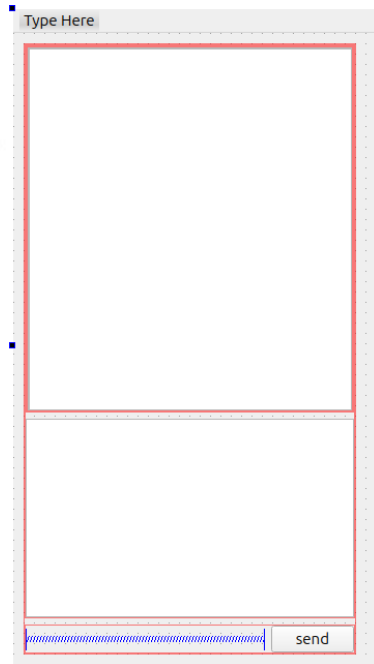
    QTextBlockFormat blockFormat;
    blockFormat.setAlignment(Qt::AlignRight);
    cursor.setBlockFormat(blockFormat);

    if(current_sender != QString("나"))
        cursor.insertText(QString("나\n"));

    current_sender = QString("나");
    cursor.insertText(QString("%1\n")
        .arg(message));

    ui->plainTextEdit->clear(); // 입력창 비우기
}

```



오른쪽의 send 버튼의 slot과 연결, 입력 부분의 text를 data에 넣어주고, WriteDatagram을 통해 send함

메신저 화면은 자신이 보낸 메시지도 뜨기 때문에 내 화면에도 메시지를 띄우도록 함

이때 이전에 메시지를 한 sender와 동일하지 않은 경우 닉네임을 표시해주도록 함

3-1-2. 키보드

Mainwindow.h

```

void on_abcBtn_clicked();
void on_defBtn_clicked();
void on_ghiBtn_clicked();
void on_jklBtn_clicked();
void on_mnoBtn_clicked();
void on_pqrsBtn_clicked();
void on_tuvBtn_clicked();
void on_wxyzBtn_clicked();
void on_spaceBtn_clicked();
void on_shiftBtn_clicked();
void on_enterBtn_clicked();
void on_backBtn_clicked();
void on_commaBtn_clicked();
void on_pointBtn_clicked();
void finalizeChar();
private:
    Ui::MainWindow *ui;

    // 헬퍼 함수
    void handleKey(const QString &keyId);

    QTimer *Timer;
    QMap<QString, QStringList> keyMap;
    QString lastPressedKey;
    int currentIndex;
    bool isUppercase;

    QList<QPushButton*> alphaButtons;
};

```

Key마다 각각 slot 함수와 버튼의 문자가 들어있는 QMap, 키입력을 도와줄 함수와 변수를 선언

Mainwindow.cpp

```

keyMap[".,?!"] = QStringList{".", ",", "?", "!"};
keyMap["abc"] = QStringList{"A", "B", "C"};
keyMap["def"] = QStringList{"D", "E", "F"};
keyMap["ghi"] = QStringList{"G", "H", "I"};
keyMap["jkl"] = QStringList{"J", "K", "L"};
keyMap["mno"] = QStringList{"M", "N", "O"};
keyMap["pqrs"] = QStringList{"P", "Q", "R", "S"};
keyMap["tuv"] = QStringList{"T", "U", "V"};
keyMap["wxyz"] = QStringList{"W", "X", "Y", "Z"};

Timer = new QTimer(this);
Timer->setInterval(800);
connect(Timer, &QTimer::timeout, this, &MainWindow::finalizeChar);

alphaButtons = {
    ui->abcBtn, ui->defBtn, ui->ghiBtn,
    ui->jklBtn, ui->mnoBtn, ui->pqrsBtn,
    ui->tuvBtn, ui->wxyzBtn
};

```

키마다 들어있는 문자를 넣어주고, 대소문자 표기를 위해 Btn list 설정

```

void MainWindow::handleKey(const QString &keyId)
{
    Timer->stop(); // 새 입력이 왔으므로 기존 타이머 중단

    if (lastPressedKey == keyId) {
        // 같은 버튼을 눌렀을 때
        currentIndex = (currentIndex + 1) % keyMap[keyId].size();

        QString targetChar = keyMap[keyId][currentIndex];
        if (!isUppercase) targetChar = targetChar.toLower();

        ui->plainTextEdit->textCursor().deletePreviousChar();
        ui->plainTextEdit->insertPlainText(targetChar);
    } else {
        finalizeChar();

        lastPressedKey = keyId;
        currentIndex = 0;

        QString targetChar = keyMap[keyId][currentIndex];
        if (!isUppercase) targetChar = targetChar.toLower();

        ui->plainTextEdit->insertPlainText(targetChar);
    }

    Timer->start();
}

```

List에 저장된 문자열이 순서대로 나옴, 같은 버튼을 누를 때 문자가 바뀌는 것처럼 연출하기 위해 지우고 새로운 문자를 추가함

```
void MainWindow::on_shiftBtn_clicked()
{
    isUppercase = !isUppercase;

    for (QPushButton *btn : alphaButtons) {
        if (!btn) continue;

        QString text = btn->text();
        if (isUppercase) {
            btn->setText(text.toUpper()); // 대문자로
        } else {
            btn->setText(text.toLower()); // 소문자로
        }
    }

    if (isUppercase) ui->shiftBtn->setText("▲");
    else ui->shiftBtn->setText("shift");
}
```

Shift의 경우 대문자와 소문자가 반전되도록 함

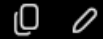
작동 영상은 hw1의 README.md 파일에 기재

3-2. AI 보고서

- UDP 구현

<https://share.gemini.google/hqpKmrloqbf6>

udp 통신을 qt로 구현하려고 하는데 <https://doc.qt.io/qt-6/qudpsocket.html> qudpsocket 사용해서 구현할건데 구현에 중요한 점만 알려줘



포트 바인딩과 주소 플래그 설정

수신(서버) 기능을 구현하려면 `bind()` 를 통해 특정 포트를 열어야 합니다. 이때 동일 장치에서 여러 프로그램이 같은 포트를 쓰거나 멀티캐스트를 활용하려면 `QAbstractSocket::ShareAddress` 와 `QUdpSocket::ReuseAddressHint` 플래그를 조합해야 포트 충돌 오류를 방지할 수 있습니다.

코드 예시:

```
udpSocket->bind(QHostAddress::Any, 7755, QUdpSocket::ShareAddress |  
QUdpSocket::ReuseAddressHint);
```

비동기 수신과 `readyRead` 루프 처리

네트워크 패킷이 도착하면 `readyRead()` 시그널이 발생합니다. 슬롯 함수 안에서는 `hasPendingDatagrams()` 가 참인 동안 `while` 루프를 돌며 `pendingDatagramSize()` 로 크기를 확인하고 `readDatagram()` 으로 데이터를 읽어와야 유실되는 데이터 없이 안정적으로 수신할 수 있습니다. 송신자의 IP와 포트 정보도 이 과정에서 함께 획득합니다.

QUdpsocket을 사용하기 위해 한 첫 질문이다.

QUdpsocket으로 UDP 구현 시 bind 기능 구현이 UDP의 핵심 코드임을 알았다.

Bind 실패 시 뒤 과정이 전부 막힌다.

QHostAddress::Any 설정은 뭐야

`QHostAddress::Any` 는 컴퓨터에 장착된 모든 네트워크 인터페이스(유선 랜, Wi-Fi 등)의 IP 주소로 들어오는 패킷을 수신하도록 지정하는 설정입니다. 내부적으로는 IPv4의 `0.0.0.0` (모든 인터페이스 바인딩)을 의미합니다.

주요 특징

- **모든 경로 수신:** PC에 여러 개의 네트워크 카드나 IP가 할당되어 있어도, 특정 IP를 가리지 않고 해당 포트로 들어오는 모든 트래픽을 받아들입니다.
- **서버/수신 측에 적합:** 메신저처럼 상대방이 내 컴퓨터의 어느 네트워크 인터페이스로 데이터를 보낼지 확실치 않거나, 일반적인 서버 프로그램을 구현할 때 기본값으로 가장 많이 사용됩니다.
- **범용성:** 개발 중인 PC의 IP가 DHCP 등에 의해 자주 바뀌더라도 IP 변경 코드 수정 없이 안정적으로 포트를 열 수 있습니다.

다른 주소 옵션과의 비교

Bind가 성공 후 통신이 되지 않아서 OPERATOR_IP 대신 QHostAddress:any를 사용할 수 있다고 하여 질문하였다.

QHostAddress를 지정하지 않고 Any로 설정하게 되면 PORT 번호가 동일한 경우 통신이 가능하게 된다.

내가 send를 눌러도 상대방 ui에 안뜨는데 이유가 뭐뭐 있을까
경우의 수

- **수신/송신 IP 및 포트 번호 불일치**

상대방의 실제 IP 주소가 아니거나, 서로 다른 포트 번호(예: 송신은 7755인데 수신은 8899로 바인딩됨)를 사용하고 있을 때 발생합니다.

확인 방법: `writeDatagram()` 의 반환값이 전송하려는 바이트 크기와 일치하는지(`-1` 을 반환하는지) 확인하고, 디버깅을 통해 수신 측과 송신 측의 포트 번호가 정확히 일치하는지 비교합니다.

- **방화벽 또는 보안 프로그램 차단**

Windows 방화벽이나 사내 보안 프로그램이 해당 포트(예: 7755)로 들어오는 인바운드 UDP 트래픽을 차단하고 있을 수 있습니다.

확인 방법: 임시로 방화벽을 끄고 테스트하거나, 방화벽 허용 목록에 해당 프로그램을 추가해 수신되는지 확인합니다.

- **수신 측 소켓 바인딩 설정 오류**

수신 프로그램에서 `bind()` 를 할 때 `QHostAddress::Any` 대신

`QHostAddress::LocalHost ( 127.0.0.1 )` 로 설정해 두면 외부 컴퓨터에서 오는 패킷을 아예 무시하게 됩니다.

확인 방법: 수신 측 소켓 바인딩 코드가 `QHostAddress::Any` 로 설정되어 있는지 소스 코드를 확인합니다.

Bind는 성공적으로 보내지는데, 송수신이 안되는 과정 중 한 질문이다.

이때 PORT 번호는 명확히 맞게 설정했기 때문에 터미널을 켜서 `ping {ip}`를 치며 tx rx가 되는지 확인하게 되었다.

스마트폰 핫스팟 환경에서 두 컴퓨터를 연결하고 있습니다.

Ping 명령어로 송수신이 안되는 것을 확인했고, 이때 양쪽 환경 모두 스마트폰 핫스팟을 사용하고 있었는데, 핫스팟을 사용하는 경우 안될 수 있다고 하여 wifi 환경으로 바꾸어주었다.

상대방에게서 message가 수신이 되는데 2개씩 돼

패킷이 2개씩 중복되어 수신되는 현상은 주로 `readyRead` 시그널이 중복으로 연결되었거나, 동일한 포트에 소켓이 여러 개 생성되어 바인딩되었을 때 발생합니다.

- **connect 함수 중복 호출 확인**

생성자나 특정 함수 안에서 `connect(text_socket, &QUdpSocket::readyRead, ...)` 코드가 의도치 않게 여러 번 실행되고 있는지 확인하세요. Qt는 동일한 시그널과 슬롯의 연결을 중복 방지하지 않고 호출된 횟수만큼 누적시키기 때문에, 패킷이 올 때마다 `udp_read()` 함수가 동시에 여러 번 실행되어 메시지가 두 번 이상 찍히게 됩니다.

- **소켓 객체 및 바인딩 중복 실행 확인**

`text_socket = new QUdpSocket(this);` 와 `bind()` 코드가 버튼 클릭 이벤트나 반복문 등 안쪽에 잘못 들어가 있어서, 실행 중에 소켓이 추가로 생성되고 포트가 여러 개 열렸는지 확인하세요. 소켓이 여러 개면 같은 포트로 들어오는 패킷을 각 소켓이 전부 받아 처리하기 때문에

송신은 하나로 가는데, 수신이 2개씩 와서 디버깅을 했던 질문이다.

여러 차례 디버깅 결과 내 코드가 아닌 상대방 코드에서 Tx를 2번씩 보내고 있어서 생긴 문제였다.

`ui->textEdit->setAlignment(Qt::AlignLeft);`

이거 때문에 옮겨지는 것처럼 보일수도 있나

네, 맞습니다! `ui->textEdit->setAlignment()` 때문에 기존에 출력되어 있던 메시지들의 정렬 방향이 의도치 않게 바뀌거나 밀려 보이는 현상이 발생할 수 있습니다.

- **원인:** `QTextEdit::setAlignment()` 은 현재 커서가 위치한 단락(Paragraph)의 정렬 방식을 바꾸는 함수입니다. `append()` 를 사용해 새 글을 추가하는 과정에서 커서 위치나 타이밍이 어긋나면, 예전에 출력했던 이전 메시지들의 정렬까지 한꺼번에 덩달아 바뀌어 버리거나 화면이 밀려 보일 수 있습니다.
- **해결 방법:** 채팅창처럼 주고받는 메시지마다 좌우 정렬을 다르게 하려면, 전체 컨트롤의 정렬을 건드리는 `setAlignment` 대신 `QTextBlockFormat` 을 사용하여 추가하려는 메시지 블록에만 개별적으로 정렬을 적용해야 안전합니다.

aligenment 설정이 좌우로 움직일 때 기존에 있던 마지막 문단을 끌고 가는 현상이 있어서 질문 하였다. SetAlignment 옵션 대신 새로운 메시지에 대해서 alignment를 정렬하는 방식으로 바꾸어서 개선하였다.

- 키보드 구현

c++기반 예제

오전 3:33

```
}

void ChunjiinKeyboard::initUi() {
    setWindowTitle("C++ 영어 천지인 자판");
    resize(350, 450);

    QVBoxLayout *mainLayout = new QVBoxLayout(this);

    // 텍스트 출력창 생성
    display = new QLineEdit(this);
    display->setReadOnly(true); // 마우스 클릭 입력만 허용
    display->setStyleSheet("font-size: 24px; padding: 10px;");
    mainLayout->addWidget(display);

    // 3x4 그리드 레이아웃 구조 정의
    struct ButtonInfo {
        QString displayName;
        QString keyId;
        int row;
        int col;
    };

    QList<ButtonInfo> buttons = {
        {"1\n.,!", "1", 0, 0}, {"2\nABC", "2", 0, 1}, {"3\nDEF", "3", 0, 2},
        {"4\nGHI", "4", 1, 0}, {"5\nJKL", "5", 1, 1}, {"6\nMNO", "6", 1, 2},
        {"7\nPQRS", "7", 2, 0}, {"8\nTUV", "8", 2, 1}, {"9\nWXYZ", "9", 2, 2},
        {"Space", "Space", 3, 0}, {"0", "0", 3, 1}, {"←", "Back", 3, 2}
    };

    QGridLayout *gridLayout = new QGridLayout();
```

💡 핵심 추가 요소 설명

1. `chunjiinTimer ( QTimer* )`

- 버튼을 누르고 특정 시간(예: 800ms) 동안 대기하면 현재 글자를 최종 확정하기 위해 추가되었습니다.

2. `keyMap ( QMap<QString, QStringList> )`

- 각 버튼("2", "3" 등)에 할당된 알파벳 묶음을 저장합니다. (예: "2" ➡ ["A", "B", "C"])

3. `lastPressedKey / currentIndex`

- 같은 버튼을 연속으로 누르고 있는지 판별하고, 현재 출력 중인 문자가 몇 번째 알파벳인지 추적합니다.

4. `isUppercase ( bool )`

- `on_shiftBtn_clicked()` 를 눌렀을 때 대문자/소문자를 토글하여 입력 결과를 변화시키기 위한 상태 변수입니다.

5. `handleChunjiinKey(const QString &keyId)`

- 개별 버튼 슬롯(`on_abcBtn_clicked` 등) 내부에서 중복되는 멀티탭 처리 로직을 하나로 모아 처리해 주는 편리한 내부 함수입니다.

이제 이 헤더 파일에 맞춰 `mainwindow.cpp` 파일에서 생성자 초기화 및 버튼 슬롯 함수들을 구현해 주시면 됩니다. 다음 단계로 **.cpp 파일의 구현 예시(소스 코드)** 작업이 필요하신가요?

```
// [추가] 천지인 로직을 위한 헬퍼 함수
void handleChunjiinKey(const QString &keyId);

// [추가] 천지인 입력 데이터 구조 및 상태 변수
QTimer *chunjiinTimer;           // 글자 확정용 타이머
QMap<QString, QStringList> keyMap; // 버튼별 문자 리스트 데이터
QString lastPressedKey;           // 마지막으로 누른 버튼 ID ("2", '
int currentIndex;                 // 현재 글자 배열의 인덱스
bool isUppercase;                 // 대소문자 전환 상태 (Shift 기능동
};
```

천지인 구현 예제가 없는 것 같아 AI한테 물어보았다.

천지인 자판 영어 버전 C++ 예제를 통해 .ui파일에 만들어놓은 버튼과 연결되도록 설정하였다.